

Preparando a API para Integração com Frontend

Uma API pronta para produção não deve apenas funcionar — ela deve ser fácil de consumir. Neste material, você aprenderá a configurar CORS, padronizar respostas, organizar endpoints e estruturar o fluxo de autenticação para integração com qualquer frontend moderno.

BACKEND → FRONTEND

NESTJS

REST API

Objetivos de Aprendizagem

Ao final deste material, você será capaz de dominar os seguintes tópicos:

1

Configurar CORS

Configurar corretamente o CORS para aplicações frontend sem comprometer a segurança.

2

Respostas Consistentes

Estruturar respostas previsíveis para facilitar o consumo da API por qualquer cliente.

3

Fluxo de Autenticação

Compreender o fluxo completo de autenticação JWT entre frontend e backend.

4

Organizar Endpoints

Organizar endpoints seguindo padrões REST pensando na experiência do consumidor.

5

Variáveis de Ambiente

Utilizar variáveis de ambiente para diferentes ambientes de execução.

6

Integração Multiplataforma

Preparar a API para integração com React, Angular, Vue ou Flutter.

Mudando a Perspectiva: Backend → Frontend

Até agora desenvolvemos a API sob a ótica do backend. Agora precisamos pensar como um frontend enxerga nossa aplicação.

O que o frontend *não* conhece

- Services e Repositories internos
- DTOs e estruturas internas
- Banco de dados e queries
- Regras de implementação
- Algoritmos e bibliotecas usadas

O que o frontend *conhece*

- **URL** do endpoint
- **Método HTTP** (GET, POST...)
- **Headers** da requisição
- **Body** enviado
- **Resposta** recebida

Por isso uma API deve ser projetada para ser fácil de consumir — a complexidade interna deve ser completamente invisível para o cliente.

O que o Frontend Realmente Precisa?

Considere um fluxo de login. O frontend não precisa saber como o JWT foi criado, qual algoritmo foi utilizado ou como o bcrypt validou a senha. Ele precisa apenas de uma resposta clara e objetiva.

Resposta mínima

```
{
  "access_token": "eyJhbGciOi..."
}
```

Resposta completa (recomendada)

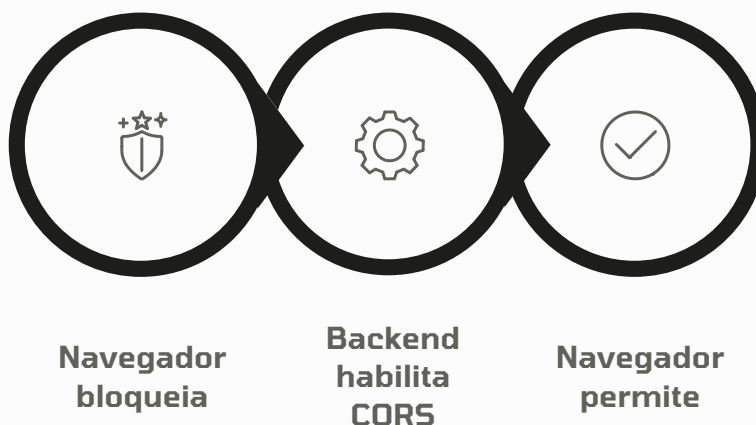
```
{
  "access_token": "eyJhbGciOi...",
  "user": {
    "id": 1,
    "name": "Maria",
    "role": "admin"
  }
}
```

Quanto mais simples e previsível a resposta, melhor a experiência do desenvolvedor frontend e menor a chance de erros de integração.

Configurando o CORS

CORS (Cross-Origin Resource Sharing) é um mecanismo de segurança dos navegadores que controla quais origens externas podem fazer requisições a um servidor. É uma política implementada pelo navegador — e não pelo servidor — para proteger os usuários de requisições maliciosas entre domínios diferentes.

Por padrão, o navegador bloqueia requisições entre origens diferentes — mesmo que frontend e backend rodem na mesma máquina.



O conflito de origens é um dos erros mais comuns na integração frontend-backend. Mesmo rodando localmente, portas diferentes já configuram origens distintas.

Origens diferentes

Frontend:
<http://localhost:5173>

Backend:
<http://localhost:3000>

Configuração no main.ts

```
app.enableCors({  
  origin: [  
    'http://localhost:5173'  
  ],  
  credentials: true,  
});
```

Com essa configuração, o navegador permitirá a comunicação entre as duas origens listadas explicitamente.

Evitando CORS Liberado para Tudo

Durante o desenvolvimento é comum encontrar configurações permissivas que liberam todas as origens. Embora funcionem, não são recomendadas para produção.

✗ Evite em produção

```
app.enableCors({  
  origin: true  
});  
  
// ou simplesmente:  
app.enableCors();
```

✓ Prefira sempre

```
app.enableCors({  
  origin: [  
    'https://meusite.com',  
    'https://app.meusite.com'  
  ],  
  credentials: true,  
});
```

Liberar CORS para todas as origens expõe sua API a requisições de qualquer domínio, incluindo sites maliciosos. Liste sempre explicitamente as origens permitidas.

Organizando Contratos de Resposta

O frontend trabalha melhor quando as respostas possuem formatos previsíveis e consistentes em toda a aplicação. Inconsistências nos contratos são uma das principais fontes de bugs na integração.

✗ Inconsistente

```
// Endpoint A  
{  
  "id": 1,  
  "name": "Maria"  
}  
  
// Endpoint B  
{  
  "userId": 1,  
  "fullName": "Maria"  
}
```

✓ Consistente

```
// Todos os endpoints  
{  
  "id": 1,  
  "name": "Maria"  
}
```

✓ Consistente - **continuação**

Nomes consistentes reduzem erros e retrabalho em toda a aplicação.

✓ Coleções com metadados

```
{
  "items": [
    { "id": 1, "name": "Maria" }
  ],
  "page": 1,
  "limit": 10,
  "total": 100
}
```

Permite incluir paginação e metadados futuramente.

Autenticação JWT e Endpoints REST

A estrutura de autenticação e a organização dos endpoints são pilares fundamentais para uma API consumível por qualquer frontend moderno.

Retorno do Login

```
{
  "access_token": "eyJhbGciOi...",
  "user": {
    "id": 1,
    "name": "Maria",
    "email": "maria@email.com",
    "role": "admin"
  }
}
```

Isso evita chamadas adicionais logo após o login — o frontend já tem tudo que precisa.

Enviando o JWT nas Requisições

GET /users

Authorization: Bearer eyJhbGciOi...

✗ Evite verbos na URL

```
/getAllUsers  
/createUser  
/updateUser  
/deleteUser
```

✓ Padrão REST

```
GET /users  
POST /users  
PATCH /users/:id  
DELETE /users/:id
```

Quando o frontend conhece o padrão REST, novos endpoints tornam-se intuitivos — GET consulta, POST cria, PATCH atualiza, DELETE remove.

Variáveis de Ambiente e Cliente HTTP Centralizado

Boas práticas de organização no frontend garantem que a API possa ser consumida de forma segura e escalável em diferentes ambientes.



Cliente HTTP Centralizado

```
Frontend  
├── api  
│   ├── auth  
│   ├── users  
│   ├── products  
│   └── categories
```

Evita espalhar URLs pela aplicação e centraliza a lógica de comunicação.



Variáveis de Ambiente

```
# .env
VITE_API_URL=http://localhost:3000

# No código
const API_URL =
  import.meta.env.VITE_API_URL;
```

A URL pode mudar sem alterar o código-fonte da aplicação.



Múltiplos Ambientes

```
# Desenvolvimento
VITE_API_URL=
  http://localhost:3000

# Produção
VITE_API_URL=
  https://api.meusistema.com
```

O frontend continua funcionando sem modificações ao trocar de ambiente.

Checklist de Validação para Integração

Antes de iniciar o desenvolvimento do frontend, verifique se todos os itens abaixo estão devidamente configurados no backend:

Autenticação e Segurança

- ✓ Login funcionando corretamente
- ✓ JWT retornado no login
- ✓ Rotas protegidas funcionando
- ✓ Header Authorization configurado
- ✓ CORS configurado com origens restritas
- ✓ Swagger atualizado

Contratos e Organização

- ✓ Respostas consistentes em todos os endpoints
- ✓ Paginação padronizada com metadados
- ✓ URLs removidas do código-fonte
- ✓ Variáveis de ambiente configuradas
- ✓ Endpoints seguindo padrão REST
- ✓ Coleções retornando objeto com items

Com todos esses itens verificados, sua API está pronta para ser consumida por qualquer aplicação frontend moderna — React, Angular, Vue ou Flutter.

Conclusão

Uma API pronta para produção não deve apenas funcionar — ela deve ser fácil de consumir. Ao configurar corretamente o CORS, padronizar contratos de resposta, organizar endpoints e estruturar o fluxo de autenticação, criamos uma base sólida para integração com qualquer aplicação frontend moderna.

Menos Problemas

Preparação adequada reduz erros de integração e retrabalho no futuro.

Mais Velocidade

Contratos previsíveis aceleram o desenvolvimento da interface significativamente.

Melhor DX

Melhora a experiência dos desenvolvedores que utilizarão a API no dia a dia.

A complexidade do backend deve ser completamente invisível para o frontend. Uma boa API é aquela que o desenvolvedor consegue consumir sem precisar ler o código-fonte do servidor.

